PRACTICAL 1:Vaccum Cleaner

Code: class TableDrivenAgent:

```
def __init__(self):       self.table =
{           (('A', 'Dirty'),): 'Suck',
          (('A', 'Clean'),): 'Right',
          (('B', 'Dirty'),): 'Suck',
          (('B', 'Clean'),): 'Left',

          (('A', 'Clean'), ('B', 'Dirty')): 'Suck',
          (('A', 'Dirty'), ('B', 'Clean')): 'Right'
       }

       self.percepts = []

   def               perceive(self,               percept):
self.percepts.append(percept)
percept_sequence = tuple(self.percepts)
       action = self.table.get(percept_sequence, "No Action Found")
return action

# Driver program agent =
TableDrivenAgent()
print(agent.perceive(('A', 'Dirty')))   # Suck

agent = TableDrivenAgent()
print(agent.perceive(('A', 'Clean')))   # Right

agent = TableDrivenAgent()
print(agent.perceive(('B', 'Dirty')))   # Suck

agent = TableDrivenAgent()
print(agent.perceive(('B', 'Clean')))   # Left
```

PRACTICAL 2: Peas Taxi Code:

```
class TableDrivenAgent:
 def __init__(self, table):
 self.table = table

   def perceive_and_act(self, percept):
      return self.table.get(percept, "STOP_AND_WAIT")

decision_table = {
   ("road-clear", "green", "no-obstacle"): "MOVE_FORWARD",
   ("road-clear", "red", "no-obstacle"): "STOP",
   ("traffic", "green", "no-obstacle"): "SLOW_DOWN",
   ("road-clear", "green", "obstacle-ahead"): "TURN_LEFT",
   ("road-clear", "green", "pedestrian-detected"): "STOP",
   ("heavy-traffic", "green", "no-obstacle"): "REDUCE_SPEED",
}

agent = TableDrivenAgent(decision_table)

percepts = [
   ("road-clear", "green", "no-obstacle"),
   ("road-clear", "red", "no-obstacle"),
   ("traffic", "green", "no-obstacle"),
   ("road-clear", "green", "obstacle-ahead"),
   ("road-clear", "green", "pedestrian-detected"),
   ("unknown-space", "green", "no-obstacle"),
]

for p in percepts:     action =
agent.perceive_and_act(p)
   print(f"Percept: {p} -> Action: {action}")
```

Practical 3b : bfs 8 puzzle

Code: from collections import deque

```
def find_blank(state):
return state.index(0)

def get_neighbors(state):
 neighbors = []     blank =
find_blank(state)     moves =
{       "Up": -3,
       "Down": 3,
       "Left": -1,
       "Right": 1
   }

   for move, step in moves.items():
new_blank = blank + step

       if move == "Left" and blank % 3 == 0:
continue
```

```
        if move == "Right" and blank % 3 == 2:
continue

        if move == "Up" and blank < 3:
continue

        if move == "Down" and blank > 5:
continue

        new_state = list(state)

        new_state[blank], new_state[new_blank] = (
new_state[new_blank],
            new_state[blank]
        )

        neighbors.append((tuple(new_state), move))

    return neighbors

def bfs(initial, goal):     queue =
deque([(initial, [])])
    visited = set()

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal:
            return path

        if current_state in visited:
            continue

        visited.add(current_state)

        for next_state, move in get_neighbors(current_state):
if next_state not in visited:
                queue.append((next_state, path + [move]))

    return None

initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 2, 0,
    4, 5, 3,
    6, 7, 8
)

solution = bfs(initial_state, goal_state)

if solution:
    print("Shortest sequence of moves:")
print(" -> ".join(solution))     print("Number
of moves:", len(solution)) else:     print("No
solution found.")
```

PRACTICAL 4: Water Jug

Code:

```
def dfs_water_jug(cap1, cap2, target):
    start = (0, 0)
stack = [(start, [])]
    visited = set()

    def get_next_states(state):
        x, y = state
next_states = []

        next_states.append(((cap1, y), "fill jug1"))

        next_states.append(((x, cap2), "fill jug2"))

        next_states.append(((0, y), "empty jug1"))

        next_states.append(((x, 0), "empty jug2"))

        pour = min(x, cap2 - y)
        next_states.append(((x - pour, y + pour), "pour jug1->jug2"))
pour = min(y, cap1 - x)
        next_states.append(((x + pour, y - pour), "pour jug2->jug1"))

        return next_states
```

```
    while stack:
        state, path = stack.pop()

        if state in visited:
continue

        visited.add(state)

        x, y = state

        if x == target or y == target:
            return path + [f"Reached target: {state}"]

        for next_state, action in get_next_states(state):
if next_state not in visited:
                stack.append((next_state, path + [action]))

    return None

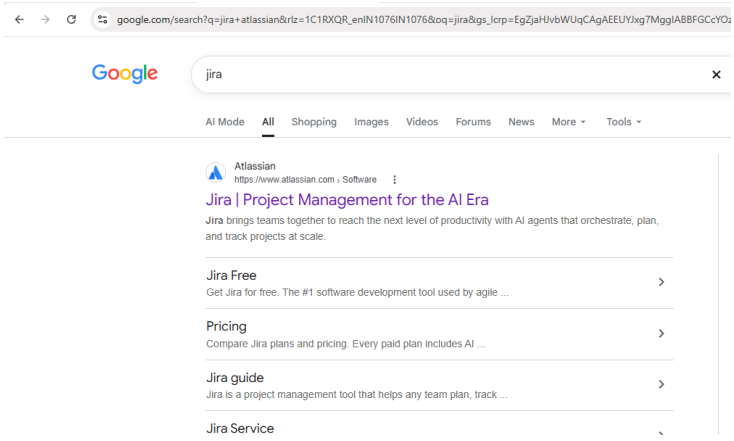
cap1,cap2=4,3 target =
2
result = dfs_water_jug(cap1, cap2, target)

if result:
    print("solution found using DFs:")
for step in result:      print(step) else:
print("no solution found:")
```

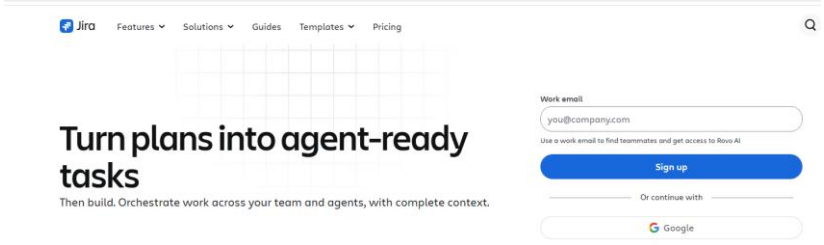
Practical 1 : Jira setup and project creation

A.create a jira cloud account and explore the dashboard
B:Create a company_manage scrum/kanban Project.
C.Configure project details

A:Create a jira cloud account and explore the dashboard
Step1: Go to Google Chrome – Search jira

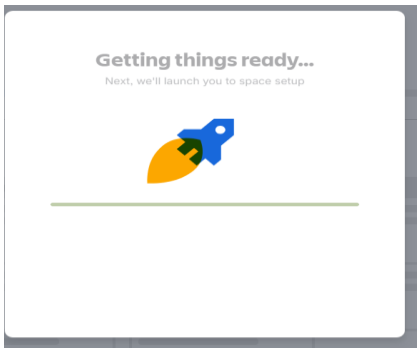


Step2: Click jira by Atlassian

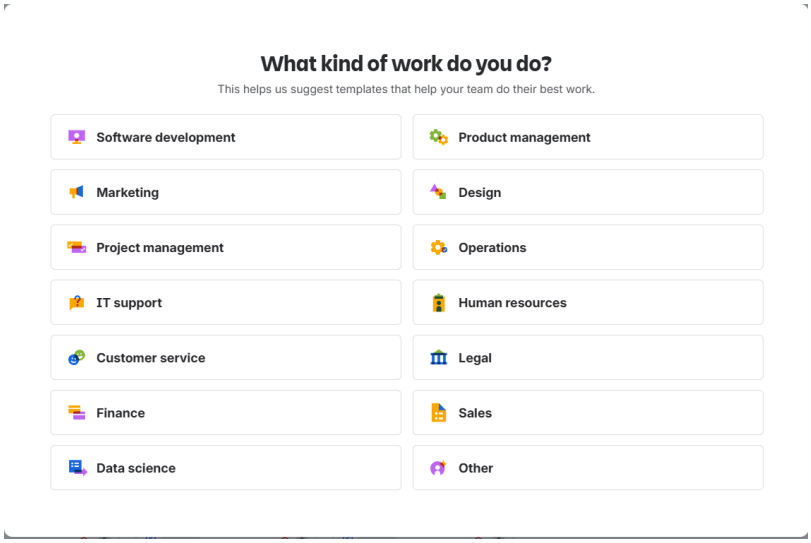


Step 3: Click on Google – Login

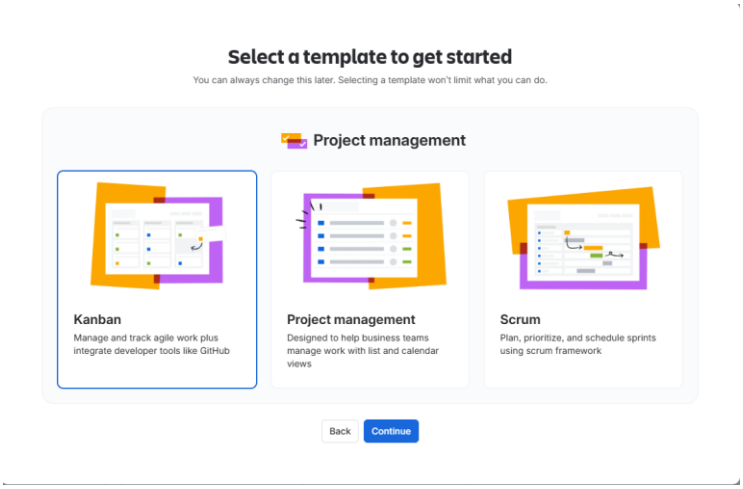
with your Gmail id – Create your account screen will appear as below-below continue.



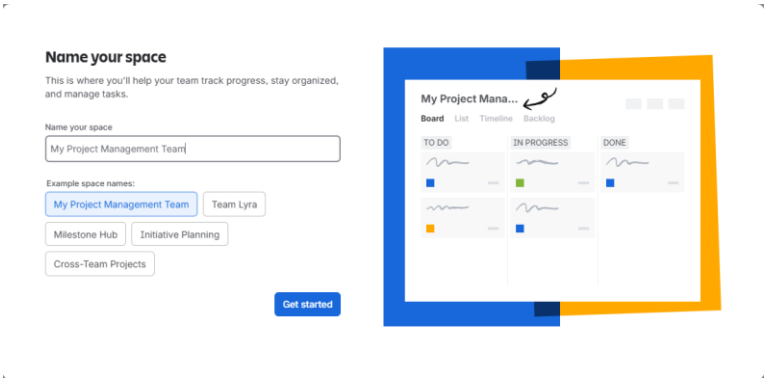
Step4: Select Project Management from the list



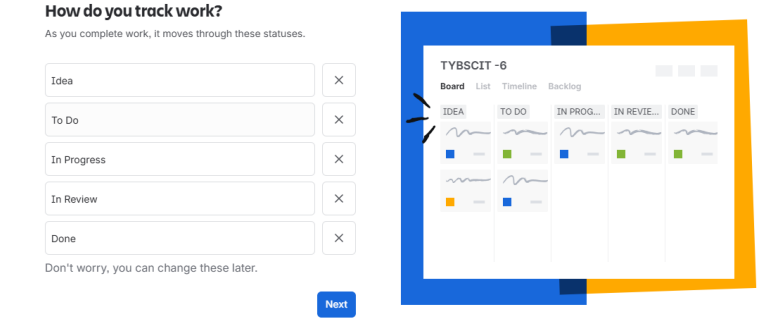
Step5: Select kanban from the list and click continue.



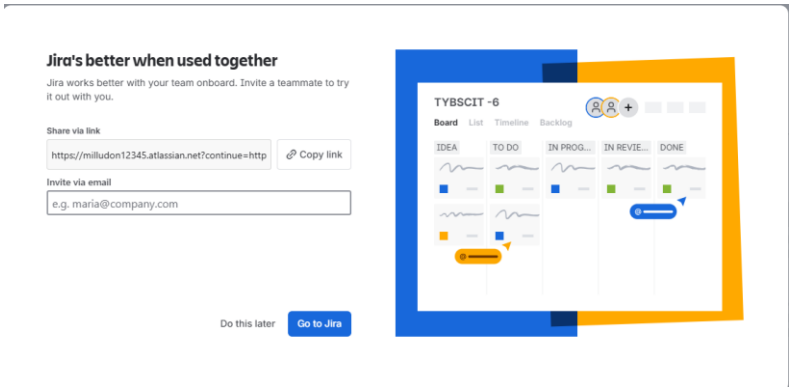
Step6: After continue – provide details as asked below and click on get started



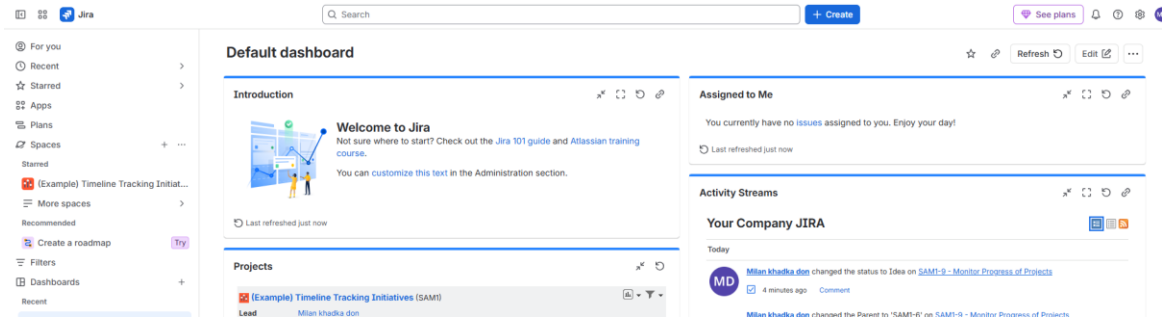
Step 7: Click Next



Step 9: Click go to jira.

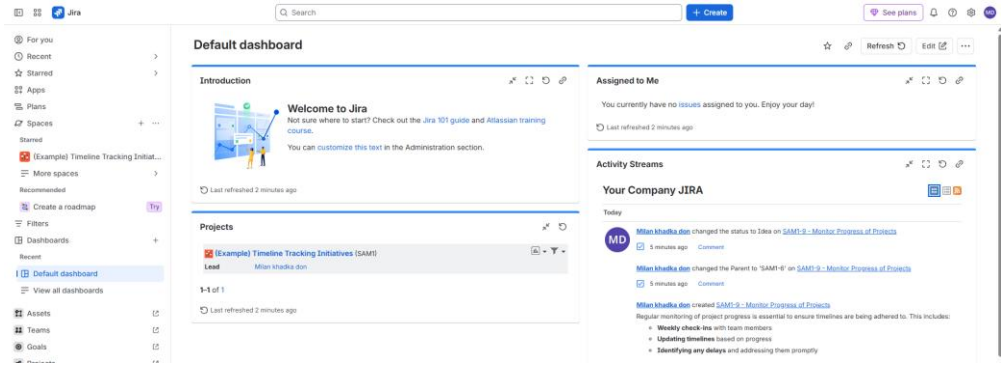


Step 9:
Dashboard will appear .

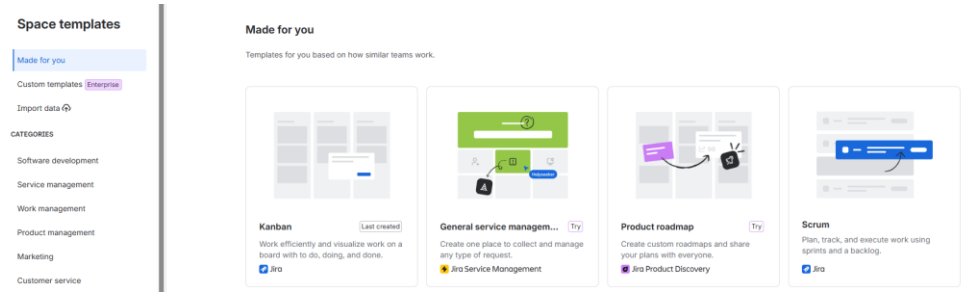


B:Create a company - managed scrum/kanban project

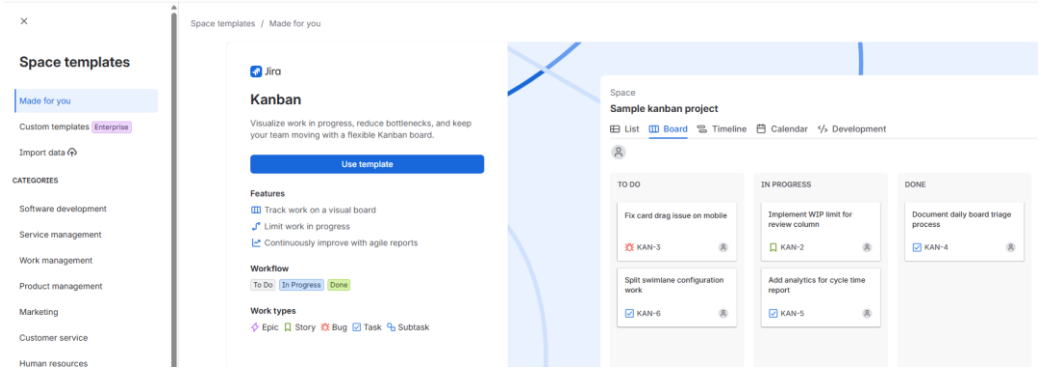
Step 1: CLICK ON spaces +



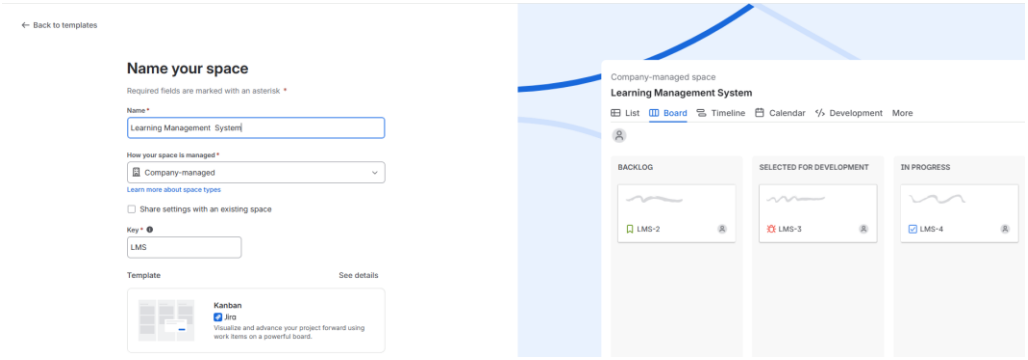
Step 2: Click on create space:



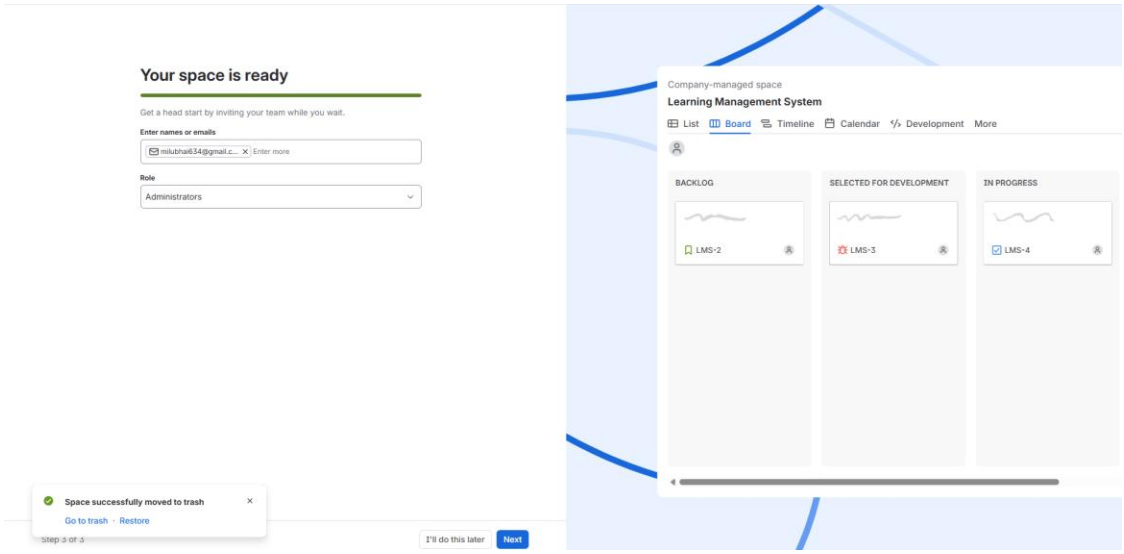
Step 3: select kanban – click use template



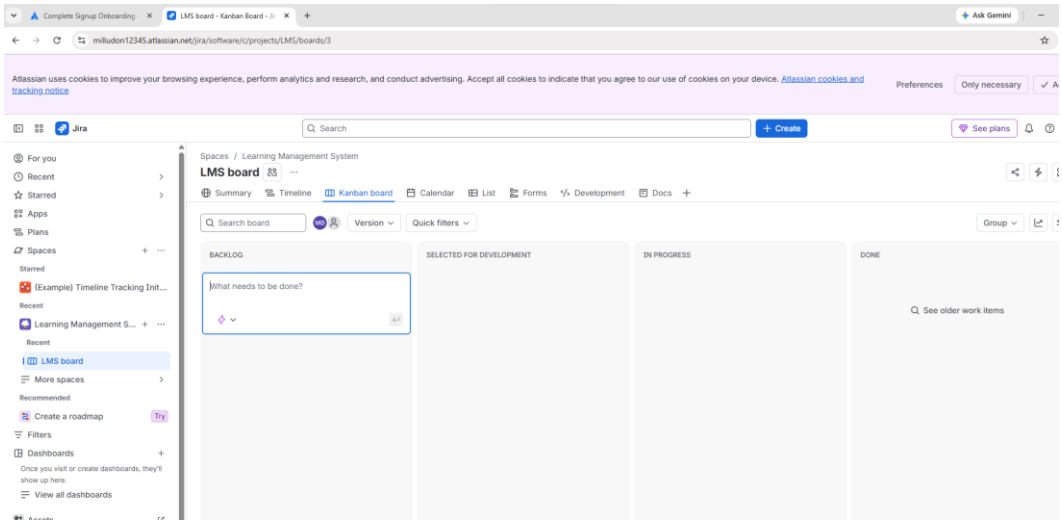
Step 4: provide details ---select company - managed in space managed option ---click Next



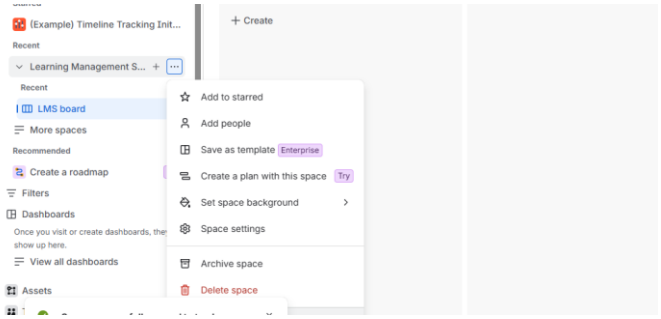
Step 5: Add emails to add members in your team or clicks I'll do this later ----click Next



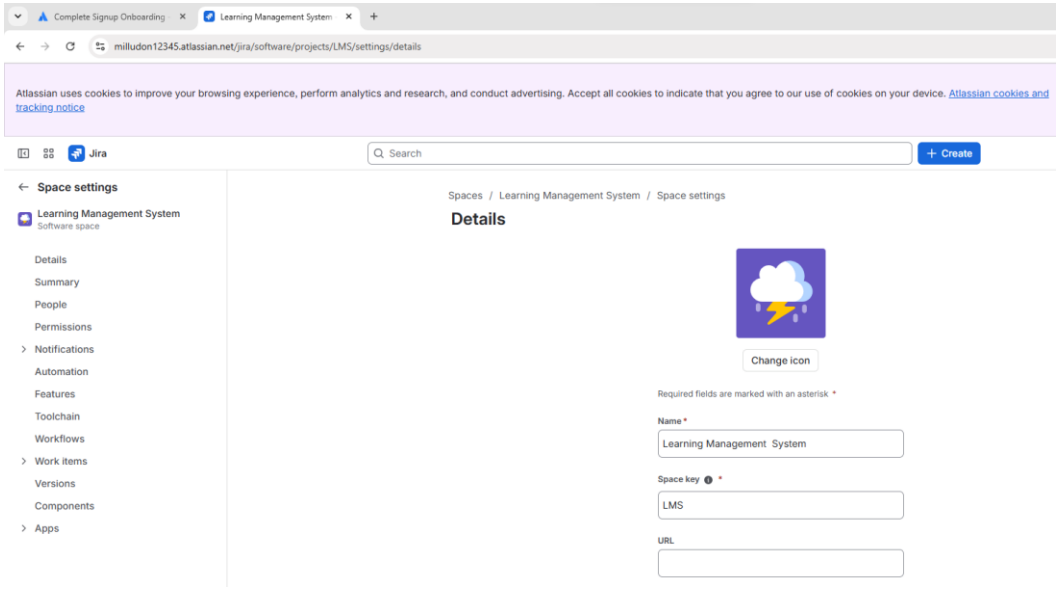
Step 6:
Following
Dashboard will
be displayed



C. Configure
project details
Step 1: click on
space Created



Step 2: CLICK space settings -- --Make necessary settings as required

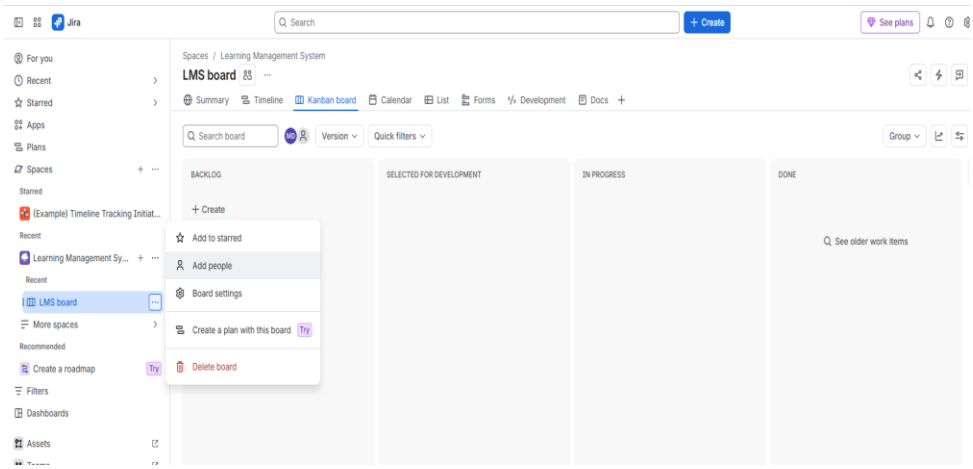


Practical 2:
Team creation and Role Assignment

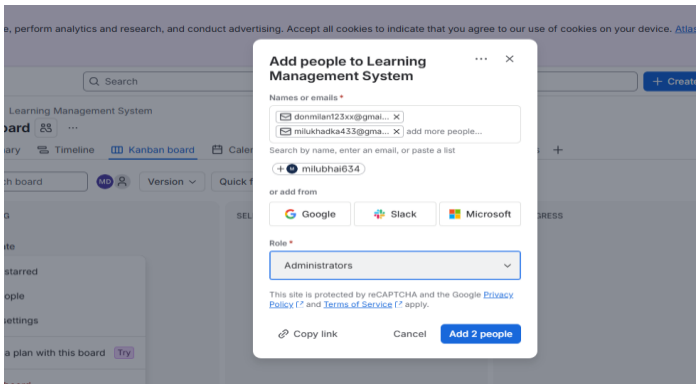
A. Add Team Members
B. Assign Roles

A. Add Team Members

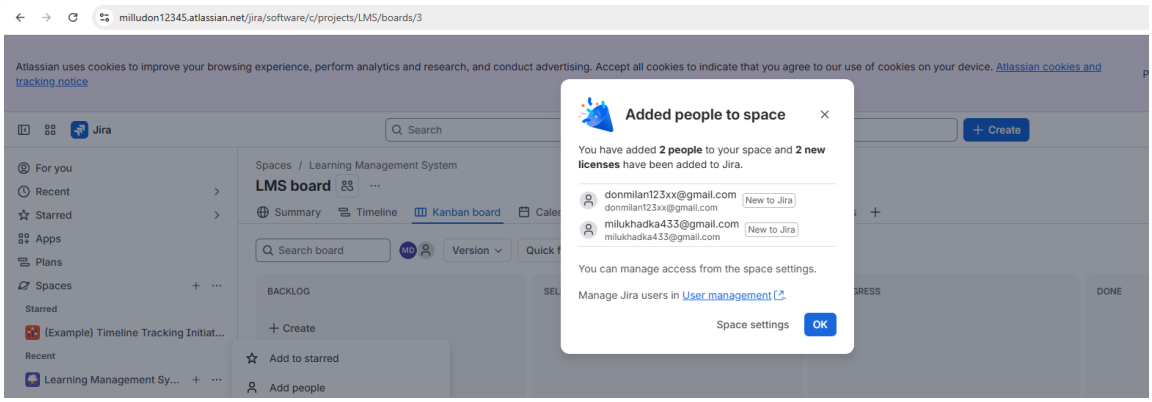
Step1: click on project –click on Add people



Step 2: In the dialogue box below—enter email to add people to your team—assign role

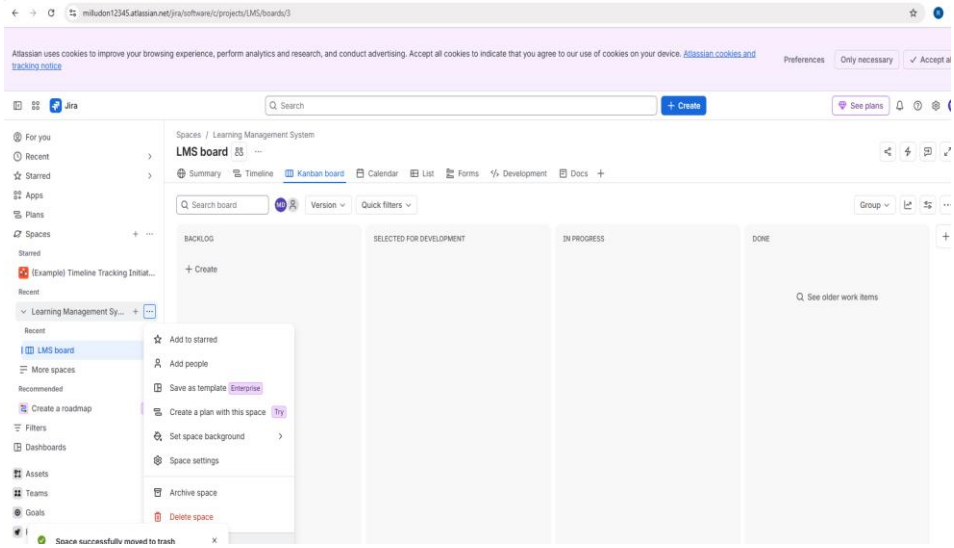


Step 3: click Add 2 person –click ok

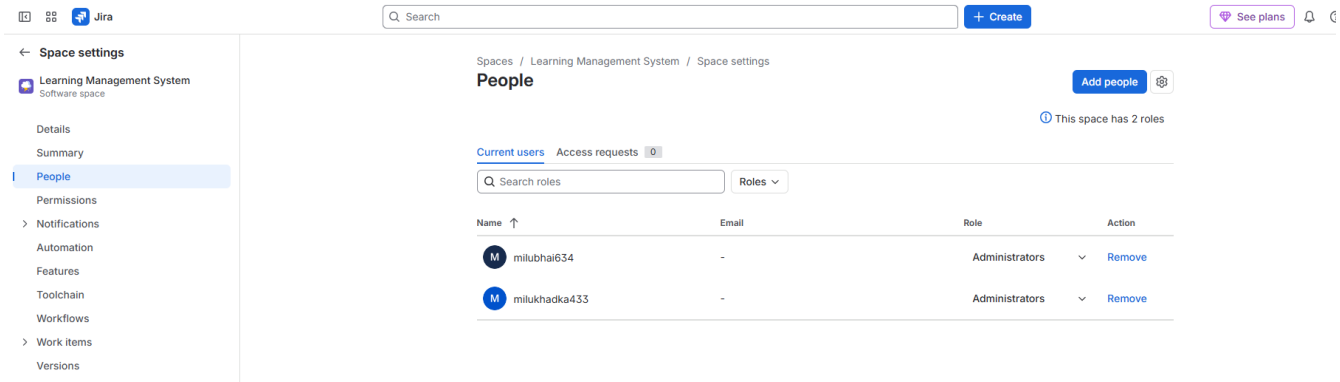


B: Assign Roles

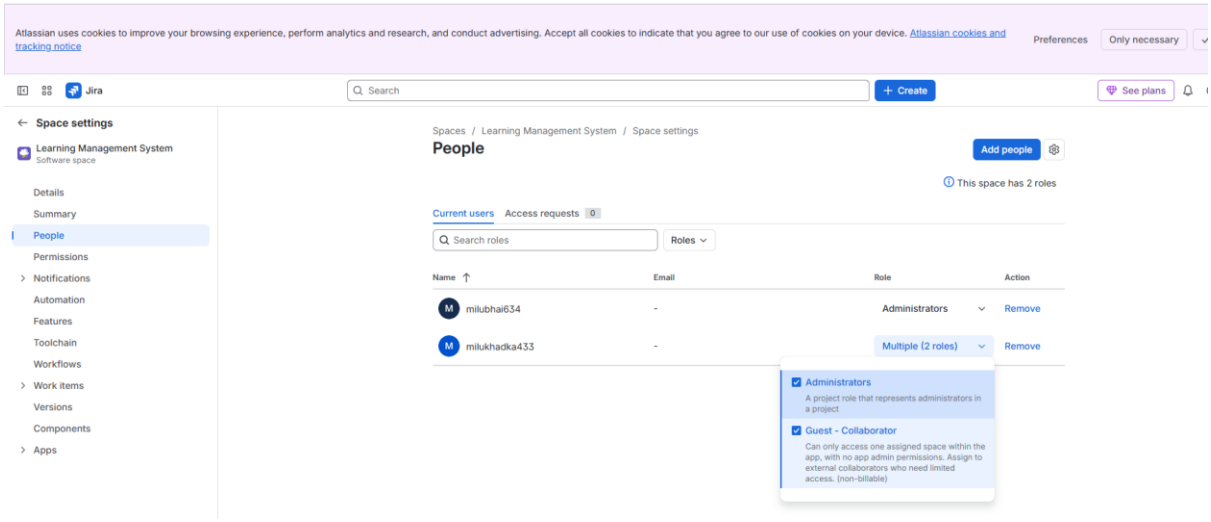
Step 1: Go to space settings



Step 2: Select people



Step 3: Select the required role from the available options

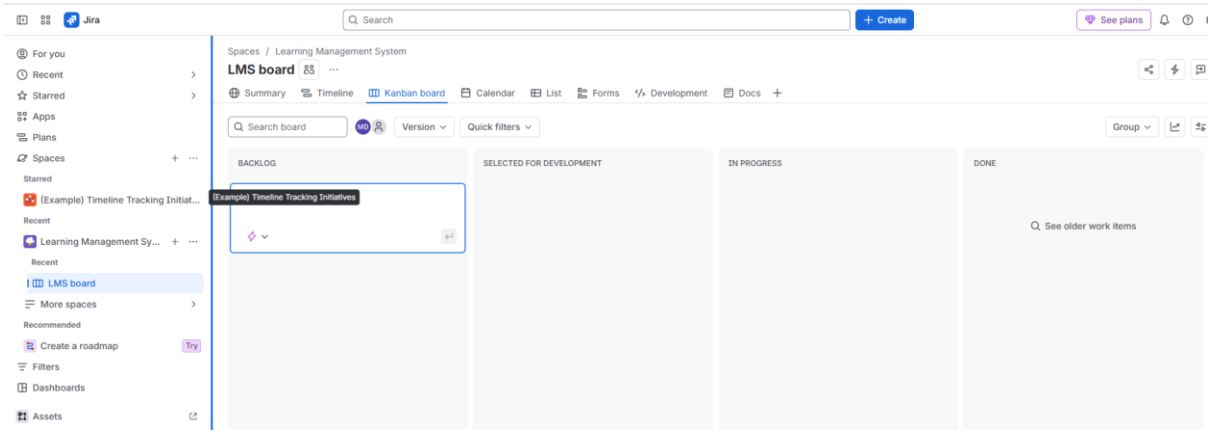

Atlassian uses cookies to improve your browsing experience, perform analytics and research, and conduct advertising. Accept all cookies to indicate that you agree to our use of cookies on your device. Atlassian cookies and tracking notice
Preferences
Only necessary
Jira
Search
Create
See plans
Space settings
Learning Management System
Software space
Details
Summary
People
Permissions
Notifications
Automation
Features
Toolchain
Workflows
Work items
Versions
Components
Apps
Spaces / Learning Management System / Space settings
People
Add people
This space has 2 roles
Current users
Access requests 0
Search roles
Roles
Name
Email
Role
Action
milubhai634
-
Administrators
Remove
milukhadka433
-
Multiple (2 roles)
Remove
Administrators
A project role that represents administrators in a project
Guest - Collaborator
Can only access one assigned space within the app, with no app admin permissions. Assign to external collaborators who need limited access. (non-billable)

Practical 3: Issues Types and Configuration

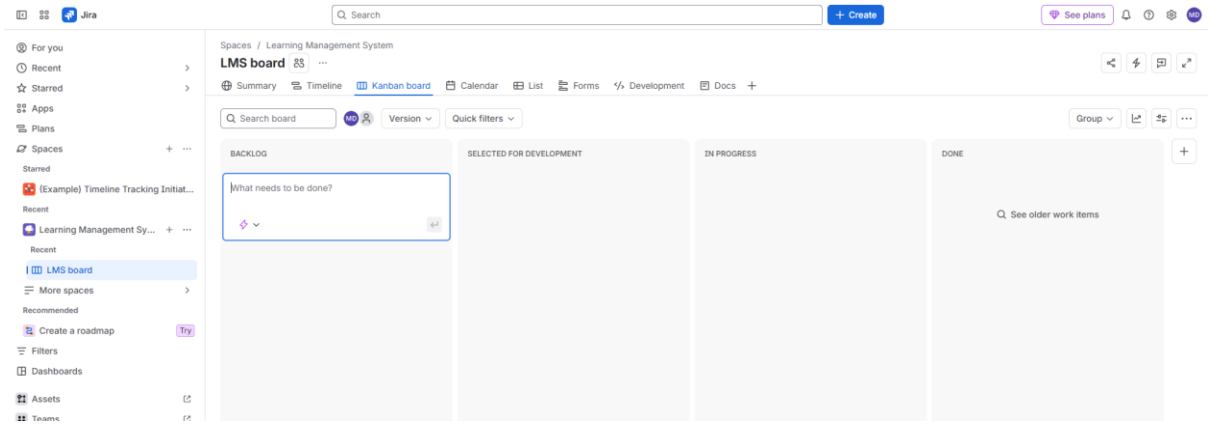
A.Create and configure Issues Types (Epic,Story,Task, Bug)

B:Create Components and Versions

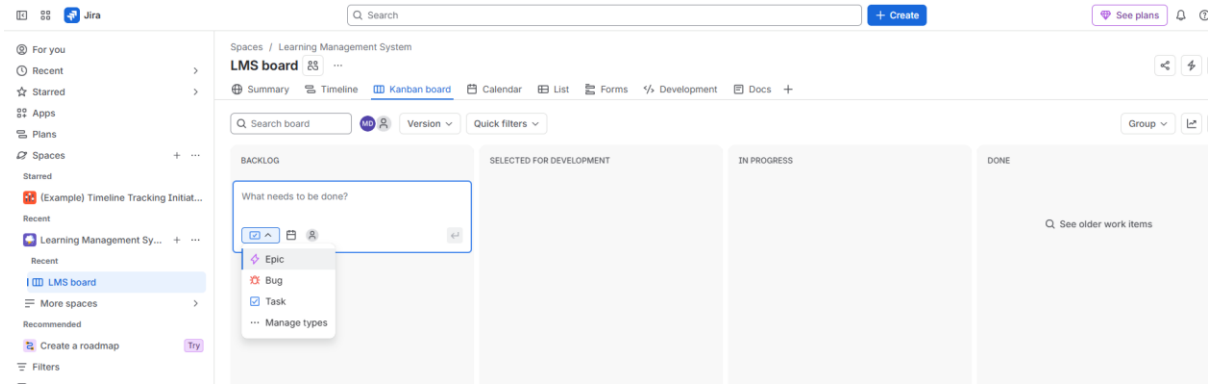
A.Create and configure Issues Types (Epic,Story,Task, Bug)



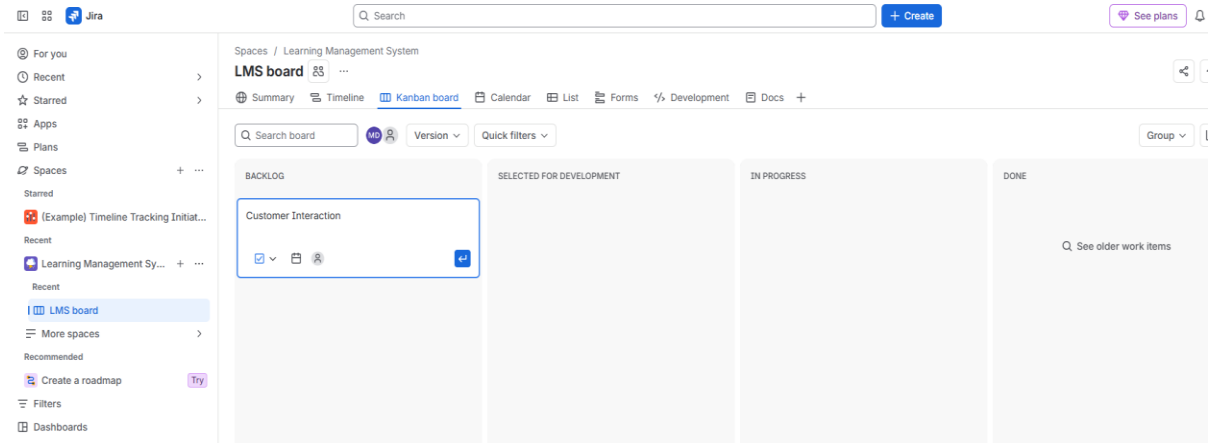
Step 1: On Dashboard—click create under Backlog



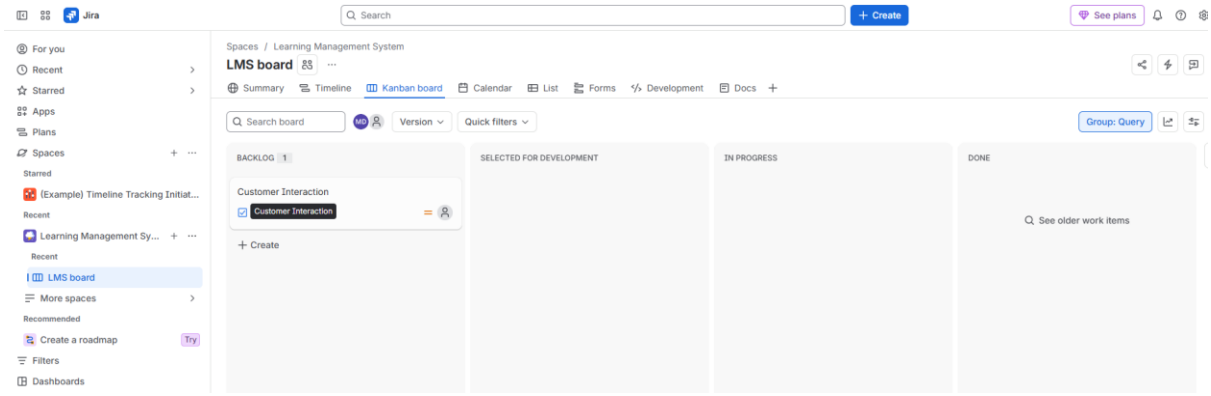
Step 2: Choose Issue type



Step 3: Enter the task—select type as task

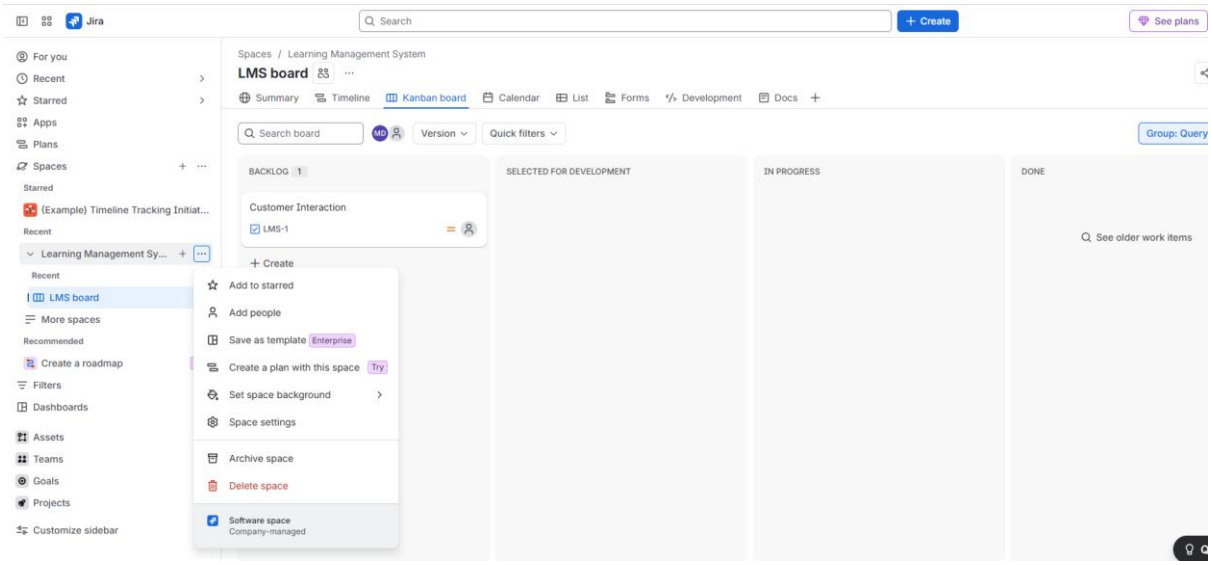


Step 4: Click Enter

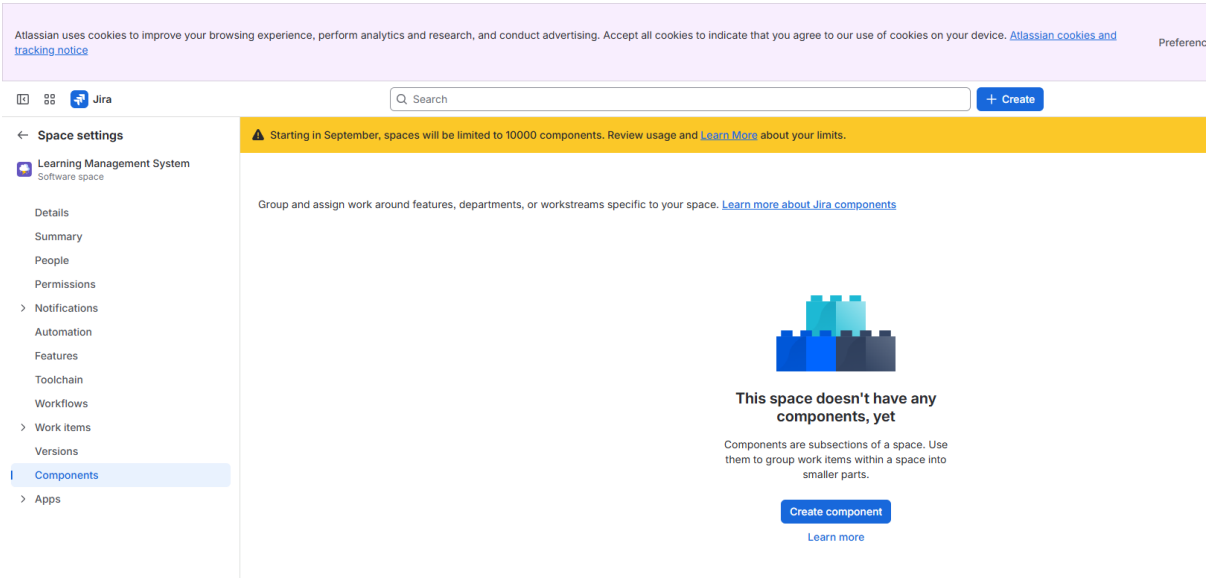


B:Create Components and Versions.

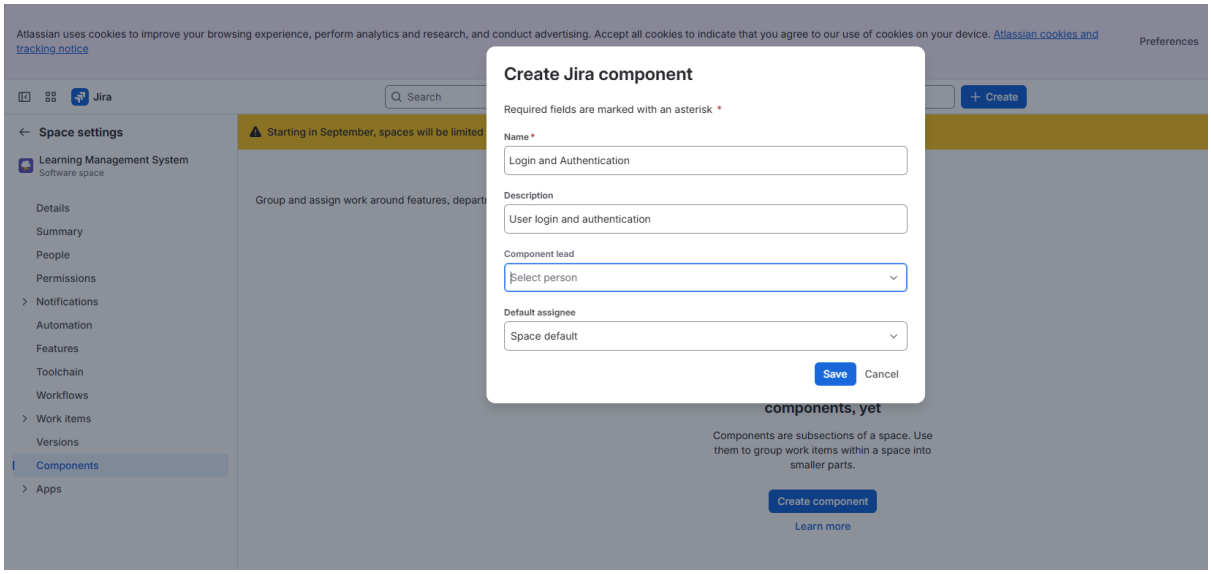
Step 1: Go to space settings



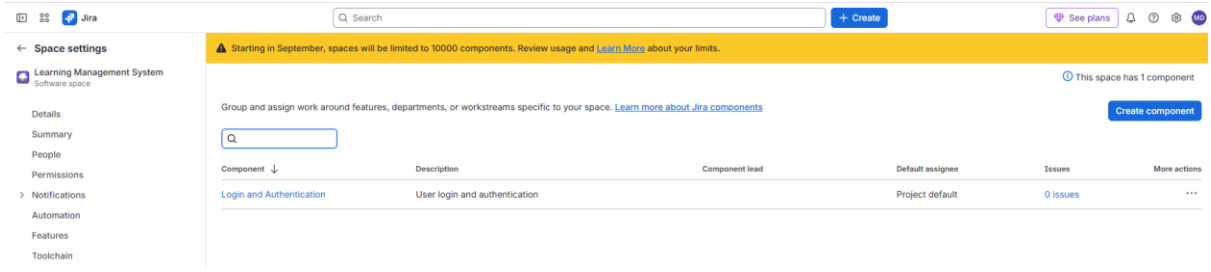
Step2: Select components – click create component



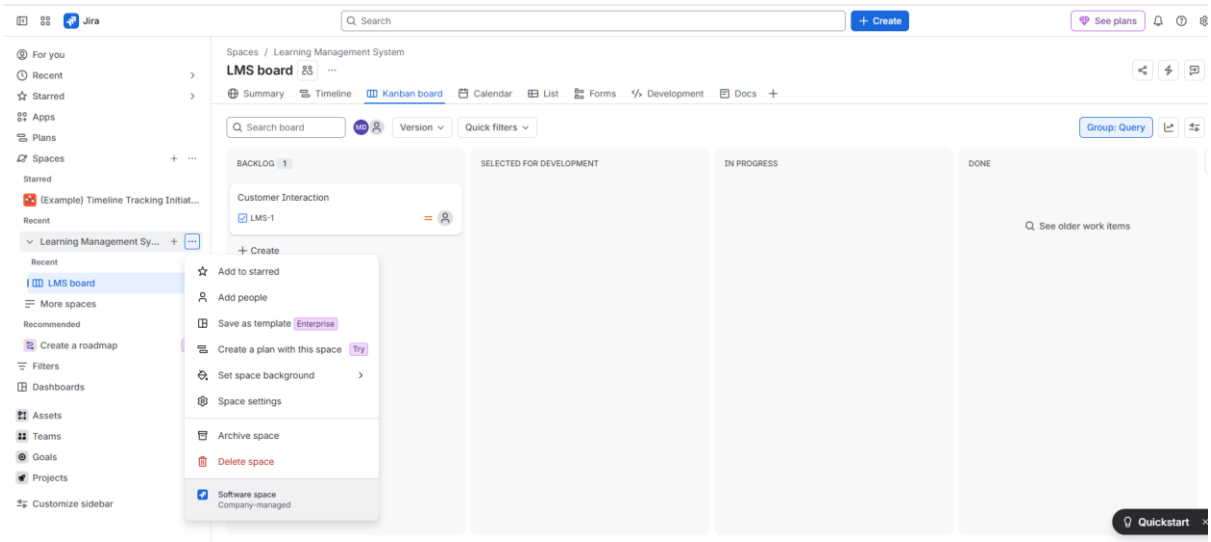
Step 3: Enter the component name ---Add a description if required –select a component lead if required



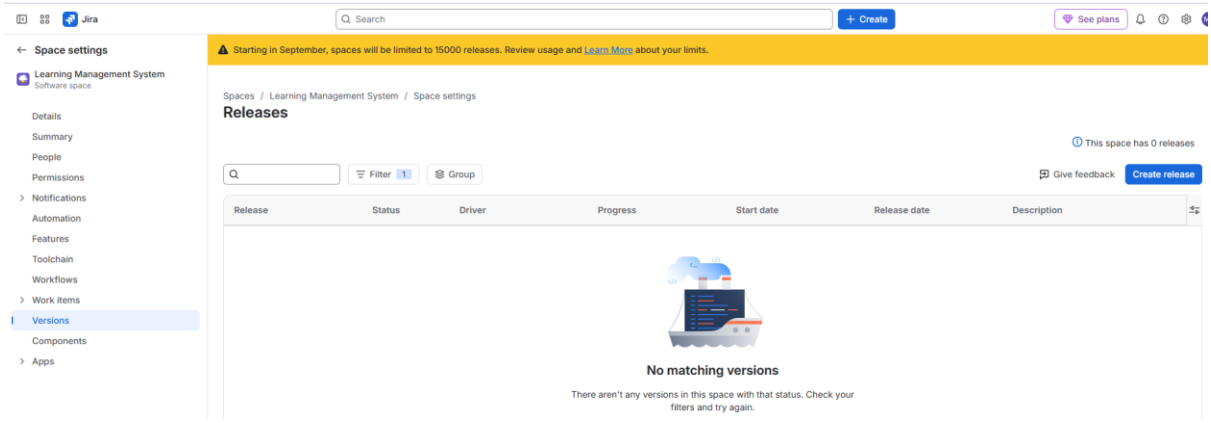
Step 4: Click Save



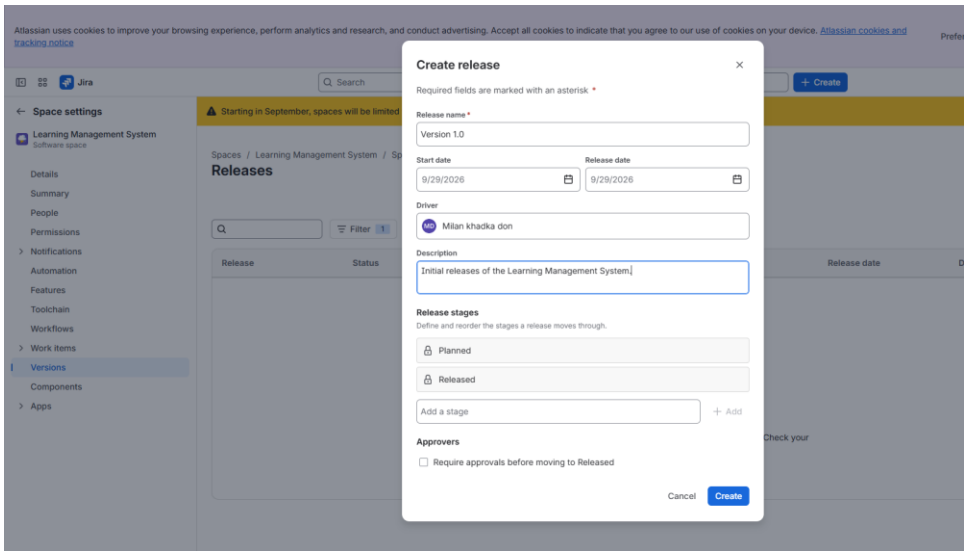
Step 5 : Go to space settings



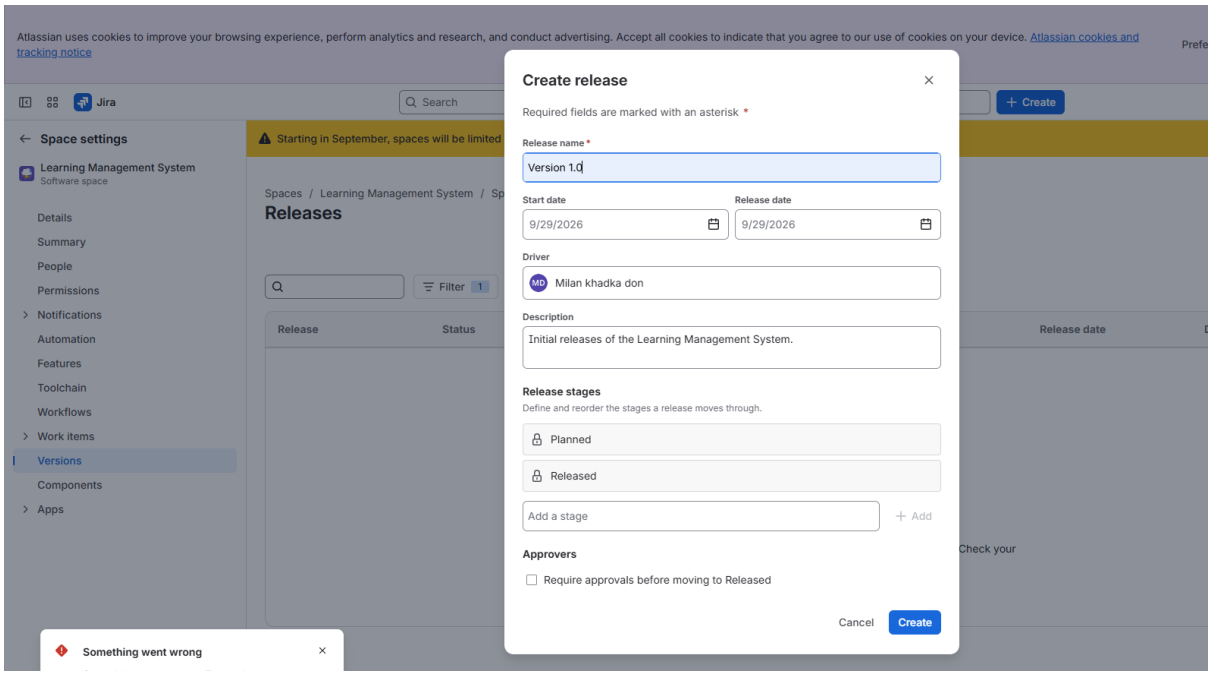
Step 6: Select versions or Releases ---click create version

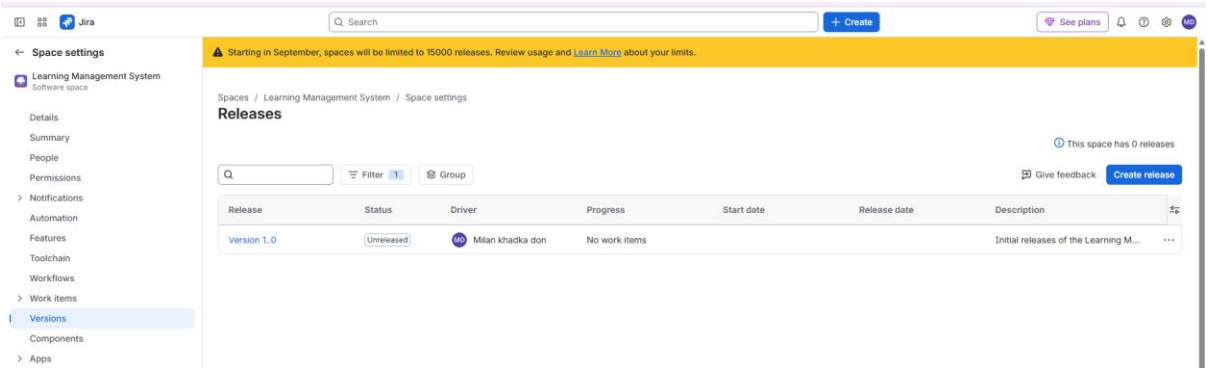


Step 7: Enter the following details



Step 8:Click Create




Jira
Search
Create
See plans
Space settings
Learning Management System
Software space
Details
Summary
People
Permissions
Notifications
Automation
Features
Toolchain
Workflows
Work items
Versions
Components
Apps
Starting in September, spaces will be limited to 15000 releases. Review usage and Learn More about your limits.
Spaces / Learning Management System / Space settings
Releases
This space has 0 releases
Filter 1
Group
Give feedback
Create release
Release
Status
Driver
Progress
Start date
Release date
Description
Version 1..0
Unreleased
Milan khadka don
No work items
Initial releases of the Learning M...

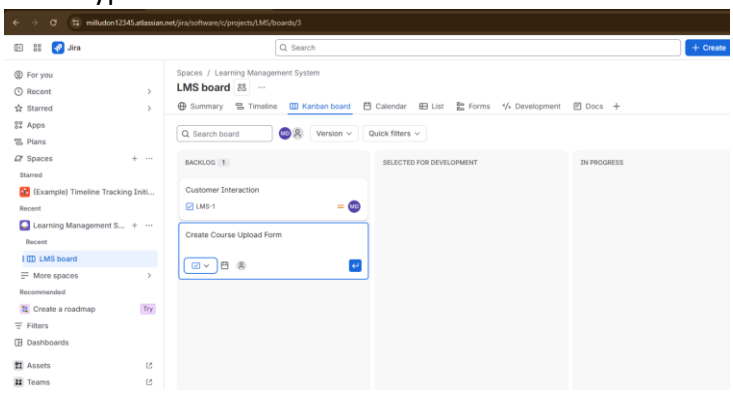
Practical 4:
Issue
Management

A:Create issues under an epic.

B .Assign issues to users and set Priority and Deu Date

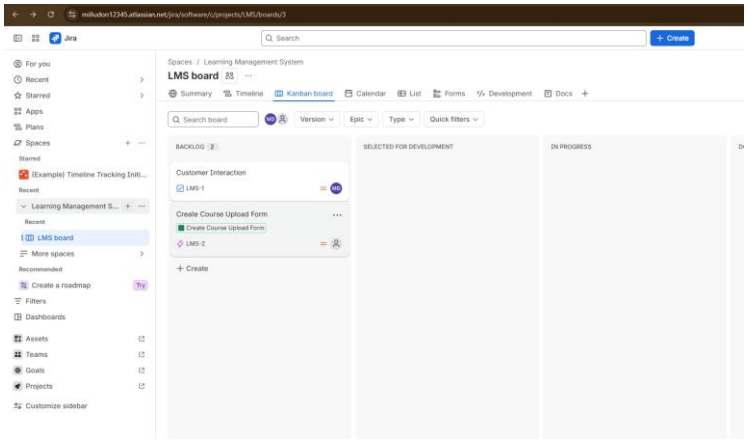
A:Create issues under an epic.

Step1: Create Issue---Select Issue Type



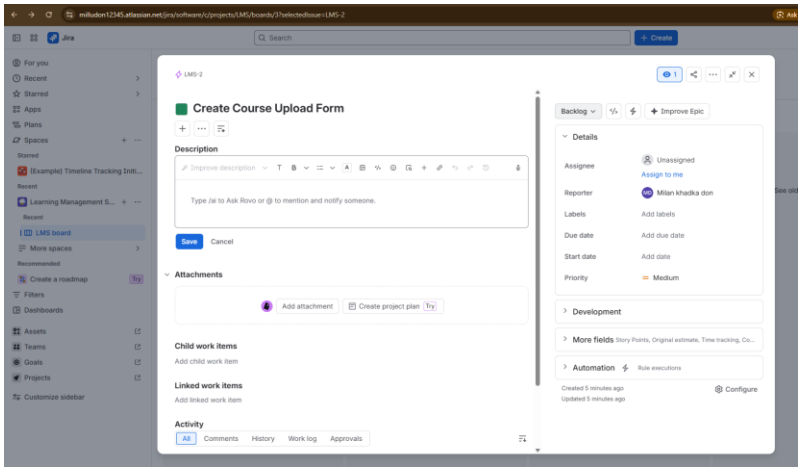
Step 2: Click epic

Step 3: Click Enter

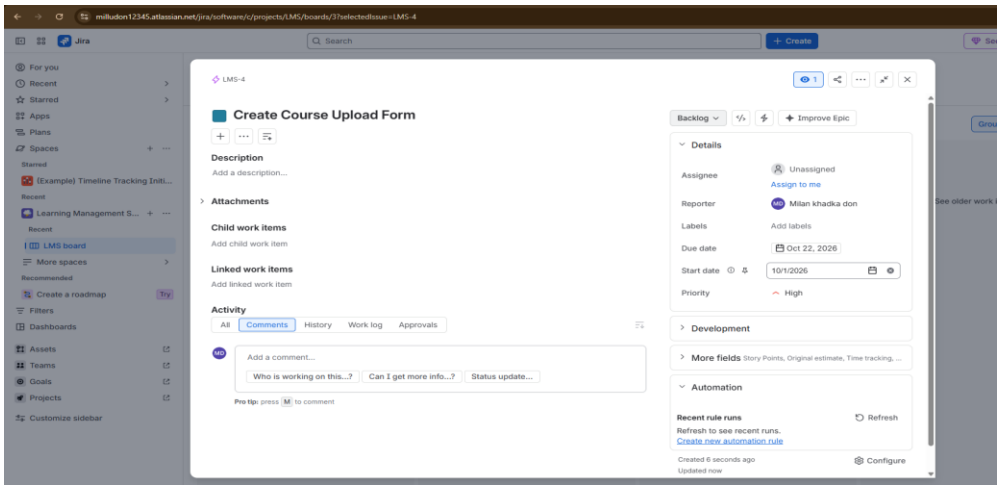


B .Assign issues to users and set Priority and Deu Date

Step 1: Open Created Issue



Step 2: Assign Issue ---Set Priority ---Set Deu Date



Step 3: Click Close ---Check Dashboard

